- **What is Java and why is it platform independent?** Java is a high-level, object-oriented programming language that is platform-independent because Java programs are compiled into bytecode, which can run on any system with a Java Virtual Machine (JVM). This makes java Write Once, Run Anywhere (WORA) language.
- **Concept of OOP** – it is a programming paradigm based on objects which represent real world entities. It includes four principles: encapsulation, inheritance, polymorphism, and abstraction. It is a paradigm to design a program using class and objects.
- A **class** is a blueprint for creating objects. It defines the properties (fields) and behaviours (methods) that an object will have.
- An **object** is an instance of a class. It represents a real-world entity with state stored in attributes and behaviour defined by methods. Class is the idea while object is the thing.
- A **method** is basically a behaviour. It is where the logics are written, data is manipulated, and all the actions are executed.
- **difference between JDK, JRE, and JVM?**
    - The JDK is a toolkit for developers,
    - the JRE provides the environment to run Java programs, and the
    - JVM is the engine, virtual machine, a runtime environment that executes the bytecode. It translates bytecode into machine code and executes it, enabling Java applications to run on any operating system with a JVM installed. At compile time, java file is compiled by Java Compiler (It does not interact with OS) and converts the java code into bytecode.
- **Java's garbage collection** automatically identifies and removes unused objects from memory, improving efficiency and preventing memory leaks**.** If a variable doesn't have reference anymore the object in the collection heap will be garbage collected.
- **Main features of Java is** platform-independent, object-oriented, robust, secure, multi-threaded, and designed for high performance.

OOP CONCEPTS

- **Inheritance** allows one class (child class) to inherit the properties and methods of another class (parent class).
    - It:
        - provides code reusability and
        - helps keep programs organized,
        - manage information in hierarchical order
        - used to achieve runtime polymorphism.
    - Multiple inheritance is not supported in java.
    - **What is the purpose of the super keyword in Java?** The super keyword refers to the immediate parent class. Super **is used in a child class to access the parent's variables, methods, or constructors when the child's own versions would normally take priority.**
    - Subclass as (derived class, child class) - The class which inherits the properties of another class. Superclass (base class, parent class). Classes whose properties are inherited

- **Polymorphism** is the ability of an object to take on many forms. It allows a single method to perform different tasks based or in different ways.

    - **Method Overloading** allows multiple methods in a class to have the same name but with different parameter lists/argument types. It is an example of compile-time polymorphism.
    - **Method Overriding** occurs when a child class provides its own implementation of a method already defined in its parent class. It is an example of runtime polymorphism

- **Encapsulation** is the practice of wrapping data and methods into a single unit (typically a class) by making fields private and providing public getter and setter methods. This controls how data is accessed and modified**.**

- **Abstraction** is a process of hiding the implementation details from the user, only the functionality will be provided to the user. In other words, the user will have information on what the object does instead of how it does it.

- **Abstract methods –** method that is declared but no implementation. If you want a class to contain a particular method but you want the actual implementation of that method to be determined by child classes, you can declare the method in the parent class as an abstract.

- **Abstract class**. It may contain both abstract methods (which lack a body) and concrete methods (which have a body).
  - a class that meant to be subclassed and cannot be instantiated.
  - a class that served as a base class for other classes.
  - a class has at least one abstract method, then the class must be declared abstract.
  - If you inherit an abstract class, you must provide implementations to all the abstract methods.

- **Interface** - is a collection of abstract class. It is a reference type that can contain abstract methods, default methods, static methods and constants. It is a contract that states that any class implementing the interface guarantees the methods in the interface will be implemented.

- **What is the difference between abstract classes and interfaces in Java?**
  - Abstract classes can have both abstract and concrete methods, whereas interfaces only have abstract methods by default.
  - Abstract classes can have constructors and instance fields, but interfaces cannot have constructors, and their fields are implicitly static and final.
  - A class can extend only one abstract class, but it can implement multiple interfaces.

- **Difference of class and interface:**
  - Class describes the attributes and behaviors of an object. And an interface contains behaviors that a class implements.
  - All methods are abstract.
  - interface is public or package scope

- **Composition and Aggregation**
  - Composition is composed of other objects that is created inside of the object. If the enclosing object is destroyed the internal object is destroyed as well.
  - Aggregation – is composed of other objects that are created outside of the object during instantiation. The lifetime of the internal objects is not dependent upon enclosing objects. If the enclosing object is destroyed the internal is still alive because it is created elsewhere.
  - Composition =- objects depends on each other.
  - Association – objects use each other but they can still exist independently

  - Composition - Holding the reference of the other class within some other class is known as composition. Implement has-a relationship in classes. Achieved by using instance variables that refers to other objects
  - Aggregation represents weak relationship whereas composition represents strong relationship. For example: bike has an indicator (aggregation) but bike has an engine (compostion).

- **What is a constructor in Java and how is it different from a method?**
  - A constructor initializes a new object and has no return type. It is invoked and called automatically when an object is created. Constructor is not inherited and should not be final.

    - 2 Types of constructors (default constructor and parameterized constructor). If there is no constructor in a class, compiler automatically creates a default constructor.
    - **Db Bb Declaration** - tells the JVM to allocate space for a reference variable.
      - – variable name with an object type. Reference variable is dog of type Dog. Dog dog.–
    - **Instantiation** – 'new' keyword is used to create the object.
    - **Initialization** – a call to a constructor that initializes the new object.

- ~~Default constructor provides the default values to the object like 0, null etc. depending on the type~~
- ~~Creating an Object. Dog dog = new Dog();~~

- A method, on the other hand, performs specific operations and must declare a return type.

❖ **What is the static keyword in Java?** The static keyword indicates that a member(field of method) belongs to the class rather than an instance of the class. Static methods and variables can be accessed without creating an object of the class.

- **Static variable**
  - Is a variable used to refer to the common property of all objects (that is not unique for each object).
  - gets memory only once in class area at the time of class loading.
- **Static method (Difference of static or instance method)**
  - Is a method that is shared by all objects (while instance method is a method that is unique for each object)
  - can be invoked without object (while instance method object is required to call instance method).

❖ **Why main method is static?** The main method is static so the JVM can call it directly without creating an instance of the class, ensuring the program has a consistent entry point.

- Whether or not you declared the main method compiler doesn't check it.
- If JVM can't find the main method it will get an error
  - Java 17: "Error: Main method not found in class MyClass, please define the main method as: public static void main(String[] args)".
  - Java 21: Java 21 introduced instance main methods, allowing programs to run without a static main method. However, if no valid entry point is found, the JVM will still throw an error indicating the absence of a proper main method.

❖ **What is reflection in Java?** Reflection allows programs to inspect and modify their own structure and behavior at runtime. It enables dynamic access to classes, methods, and fields, even if they were not explicitly known at compile time.

THREADS

What is a thread in Java and how is it different from a process?

- **A thread is an independent path of execution within a program**. It is a subset of the process or a lightweight subprocess while ***process*** is an executing instance of a program.
- **Many threads can run concurrently within a program.**
- ***Multiple threads*** within the same process share memory, while ***processes*** are independent and do not share memory

Concurrency -executing multiple tasks on the same core.
Parallelism - executing multiple tasks on multiple cores/simultaneously.
Synchronization - process that keeps all concurrent thread execution to be in sync.

❖ **What is multi-threading in Java and how is it implemented**? Multi-threading allows multiple threads to execute simultaneously, maximizing CPU utilization. It can be implemented by extending the Thread class or implementing the Runnable interface.

2 ways of making a thread in java.

1. Extending a class to Thread

```java
class Runner extends Thread {
    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println("Hello" + i);
        }

        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

2. Implementing the runnable interface and make a thread

```java
class RunnerTwo implements Runnable {
    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println("Runnable interface" + i);
        }
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

```java
public class MainThreadClass {

    public static void main(String[] args) {
        //runner1 and runner2 are objects that runs on seperate thread
        //start() it looks for the run method is used to start a newly created thread of an object
        //be mind that is you directly call run() method you are calling the method on the main thread not
creating another thread

        //Type of making a thread in java
        // extending a class to Thread
        Runner runner1 = new Runner();
        Runner runner2 = new Runner();

        runner1.start();
        runner2.start();

        //implementing the runnable interface and make a thread
        Thread thread = new Thread(new RunnerTwo());
        Thread thread2 = new Thread(new RunnerTwo());

        thread.start();
        thread2.start();


        //thread by anonymous class
        Thread thread3 = new Thread(new Runnable() {
            @Override
```

```
            public void run() {
                for (int i = 0; i < 10; i++) {
                    System.out.println("Runnable interface" + i + "-" + thread.getName());
                }

                try {
                    Thread.sleep(100);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }

            }
        });

        //thread by java 8 lambda
        Thread thread4 = new Thread(() - > {
            for (int i = 0; i < 10; i++) {
                System.out.println("Runnable lambda" + i + "-" + thread.getName());
            }

            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

        });

        //lambda using method reference class=ThreadSample method=methodReferenceMethod
        Thread thread5 = new Thread(ThreadSample::methodReferenceMethod); //this is the same for Thread
thread5 = new Thread(()-> methodReferenceMethod())

        thread3.start();
        thread4.start();
        thread5.start();

        try {
            thread3.join();
            thread4.join();
            thread5.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        //join waits the thread to finished before going back to main thread and perform succeeding actions
        //volatile public volatile boolean running = false;
        //volatile is used to prevents thread caching variables
        //Declaring a variable volatile thus guarantees the visibility for other threads of writes to that
variable.

        //synchronized
        //calling a synchronized method. allows only one thread to access or acquire the intrinsic lock at a
given time.
        //one thread will access the synchronized while the other wait to realease the intrinsic lock

        //Thread Pool
        ExecutorService execurtorService = Executors.newFixedThreadPool(2); //act like 2 workers or 2 threads

        //two thread of execurtorService will do the 5 tasks in the loop.
        //once the thread finishes the task it will start other task
        for (int i = 0; i < 5; i++) {
            execurtorService.submit(new Processor(i));
        }

        execurtorService.shutdown();
```

```
        System.out.println("All tasks are submitted"); //by calling execurtorService.shutdown();
        try {
            execurtorService.awaitTermination(1, TimeUnit.DAYS);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("All tasks are completed");

    }

    public static void methodReferenceMethod() {
        for (int i = 0; i < 10; i++) {
            System.out.println("Runnable methodReferenceMethod lambda" + i);
        }
    }
}
```

- **What is the difference between Callable and Runnable in Java?** A Runnable task does not return a result and cannot throw checked exceptions, while a Callable task returns a result and can throw checked exceptions.
- **What is a deadlock in Java?** A deadlock occurs when two or more threads are waiting for each other's resources, causing them to remain in a waiting state forever. It is a common multi-threading issue that should be avoided.
- **What is the difference between wait and sleep in Java?** wait is used in multi-threading to pause the current thread until it is notified and releases the monitor lock. sleep pauses the current thread for a specified time but does not release any locks.
- **What is the difference between synchronized and lock in Java?** Synchronized is an implicit locking mechanism that automatically acquires and releases locks, making it simpler but less flexible for handling concurrency. Lock provides explicit control over locking with features like tryLock() and lockInterruptibly(), but requires manual lock release, making it more error prone.

- ~~The synchronized keyword provides a simple mechanism to control access to critical sections. lock is a more flexible and powerful mechanism from the java.util.concurrent package, offering additional features like timed locks and interruptible locks.~~

EXCEPTIONS

Exceptions – we could handle in the program. ***What is the difference between checked and unchecked exceptions in Java***

- **<u>Checked Exceptions</u>** **(**(IOEXcpetion, SQLException)
  - must be declared in the method signature or handled using a try catch block
  - the calling method should handle the exception.
- **<u>Unchecked Exceptions</u>** (RuntimeException like Arithmetic exception)
  - do not need to be declared or explicitly caught.
  - the calling method does not need to do anything about the exception.

Error – cannot handle

User-defined Exception can be made by extending the Exception Class

```
class MyException extends Exception {
  public myException(String msg) {
    super(msg) // constructor of exception class
  }
}
```

- **What is the difference between throw and throws in Java?**
  - **throws**- suppress the exception and send it to the one who called the method. It is used in a method signature to declare the exceptions a method might throw.
    - ***Ex. Public class myMethod() throws Exception.***
  - **throw** forces to call an exception. It is used to explicitly throw an exception in the code.
    - ***Ex. throw new ArithmeticException();***
- **What is the difference between final, finally, and finalize in Java?**

- ***final*** - prevents changes. is a keyword used to declare constants, prevent method overriding, or prohibit class inheritance/ prevent a class from being subclassed.
- ***finally*** is a block that ensures execution after a try-catch block, whether or not an exception occurs.
  - You can either declare just catch or just finally or both catch and finally.
- ***finalize*** is a method called by the garbage collector before reclaiming an object's memory.

JAVA MODIFIERS

Names used for classes, variables, and methods are called **identifiers.**

Access Modifiers – Set levels for classes, variables, methods, and constructors

- Visible to the world (public).
- Visible to the package and all subclasses (protected).
- Visible to the package, (the default. No modifiers are needed.)
- Visible to the class only (private).

Access Modifiers	*Same Class*	*Same Package*	*Subclass*	*Other packages*
public	Y	Y	Y	Y
protected	Y	Y	Y	N
no access modifier	Y	Y	N	N
private	Y	N	N	N

Non-access Modifiers – final, abstract, strictfp.

- **What is the purpose of the volatile keyword in Java**? The volatile is a keyword used to ensure visibility of changes to variables across multiple threads. Any modification to volatile variable is immediately written to the main memory, making it visible to all threads.

VARIABLES and TYPES

Basic Operators:

- The Arithmetic Operators (+,-,*,/,%,++,--)
- The Relational Operators (>, < ,=, <=,>=,!=)
- The Bitwise Operators

- **What is the difference between ==** and the equals method in Java? == checks if two references point to the same object in memory, while equals checks if the values of two objects are logically equal. Use equals for value comparisons in most cases.

Variables:

- **Local Variables** = inside methods, constructors or blocks
- **Instance Variables** (Non-static Variables) = variables within a class but outside any method.
- **Class Variables** (Static Variables) =Class variables are variables declared within a class, outside any method, with the static keyword.

Primitive Data Types (all small caps)

- *byte* = 8 bit signed two's complement integer.
- *short* = 16 bit signed two's complement integer.
- *int* = 32 bit signed two's complement integer.
- *long* = 64 bit signed two's complement integer.
- *float* = 32 bit floating point
- *double* = 64 bit floating point
- *boolean* = 1 bit of information. True or False
- *char* = 16-bit Unicode character

Reference/Object Data Types - are created using defined constructors of the classes. They are used to access objects.
Wrapper Class - class that wraps the primitive data type into Object type;

- ***Autoboxing*** – java automatically converts primitive data type to corresponding Wrapper class/Object type.
  Ex. Integer myInt = 42; //declared Integer wrapper class like you declared a primitive int
- ***Unboxing*** = unwrap wrapper class to corresponding primitive data type

```
Integer numObj = Integer.valueOf(100);
// Unboxing: converting Integer to int automatically
int num = numObj
```

Casting – converts one data type to the other

- ***Cast Down*** - You have to do manual casting if you cast from higher to lower data type(cast down) like double to int. *E.g : int a = (int) 2.5; 2.5 is a double converts to int*
- ***Cast Up*** - Java cast data type from lower to higher (cast up) *E.g. byte b = 1, int ex=b;*

Mutable object is an object whose state can be modified after it is created.
Immutable object is an object whose state cannot be modified after it is created.

- **What is the difference between StringBuilder and StringBuffer in Java**?
  - String - Immutable
  - StringBuffer – mutable, slower but thread safe (synchronized). Use for multi-threaded programs
  - StringBuilder – mutable, faster but not-thread safe (not synchronized). Use for single-threaded programs

COLLECTIONS

- **What is the Java Collections Framework?** The Java Collections Framework is a set of interfaces and classes for storing and processing groups of objects. Common examples include List, Set, Map, and their implementations (like ArrayList and HashMap).

Java Arrays - are objects that store multiple variables of the same type. Though array can store primitive, array is always an object.
Java Enum -is used to define collections of constants.

- **What is the difference between array and ArrayList in Java?** An array has a fixed size, while an ArrayList can dynamically grow and shrink. Additionally, ArrayList provides many utility methods (like add and remove) for easier manipulation.
- **What is the difference between ArrayList and LinkedList in Java?**
  - ArrayList is better for storing and accessing data as it uses a dynamic array.
  - LinkedList is better for manipulating data as it uses a doubly linked list. Each has different performance characteristics based on the operation.
- **What is the difference between HashMap and Hashtable in Java?** Hashtable is synchronized (making it thread-safe but slower) it doesn't allow null key values. HashMap is not synchronized (meaning it is faster but not thread-safe) and allows null keys and values**.**
- **What is the difference between a HashSet and a TreeSet in Java?** A HashSet is unordered and faster for operations like add and remove, while a TreeSet is sorted and allows navigation in a specific order.

Generics - <T> provides compile type safety so that only single type of object can be added to a collection.

```
Class Myclass<T> {
}
//limit the type to numeric values
Class Myclass<T extends Number> {
}
```

- **What is the difference between Comparator and Comparable in Java**? ***Comparable*** is an interface used for natural ordering of objects by implementing the compareTo method. whereas ***Comparator*** is a interface for custom ordering, allowing sorting logic outside the class. Use Comparable when objects have a default sort order, and Comparator when flexibility in sorting is needed.

```java
public class Person implements Comparable<Person> {
    String name;
    Integer id;

    Person(String name, Integer id) {
        this.name = name;
        this.id = id;
    }

    @Override
    public int compareTo(Person arg0) {
        return name.compareTo(arg0.name);
    }
}

public class ListSample {
    public static void main(String[] args) {
        listSamples();
    }

    public static void listSamples() {
        //=============== LIST =======================
        System.out.println("\nLIST");

        //methods add(element),  remove(index), get(index), set(index, value);
        //stores elements by insertion order/ maintain the insertion order. it's an ordered collection
        //it is indexed
        //ArrayList(NOT thread safe) and LinkedList (NOTthread safe) and Vector(thread-safe with synchronized)
        List < Integer > lists = new ArrayList < Integer > ();
        lists.add(1);
        lists.add(5);
        lists.add(8);
        lists.add(6);
        lists.add(6); //allow duplicates

        //for Each lists [1, 5, 8, 6]
        for (Integer element: lists) {
            System.out.println(element);
        }

        List < Integer > lists2 = new ArrayList < Integer > (lists);

        lists2.addAll(lists);

        //regular for lists [1, 5, 8, 6]
        for (int i = 0; i < lists2.size(); i++) {
            System.out.println(lists2.get(i));
        }

        //sorts the list [1, 5, 6, 8]
        Collections.sort(lists);
        System.out.println(lists);

        Iterator < Integer > iterator = lists.iterator();
        //removing an item from the list in a foreach will cause a concurrentModificationException
        //concurrentModificationException happens when you remove an item while iterating/looping thru the
list
        //to solve this use Iterator which allows you to remove an item while iterating the list
        while (iterator.hasNext()) {
            Integer value = iterator.next();
            System.out.println(value);
            if (value.equals(5)) {
                iterator.remove();
            }
        }
```

```java
        //sorts the list [1, 6, 8]
        System.out.println(lists);

        //sample of list with custom object = Person
        List < Person > people = new ArrayList < Person > ();
        people.add(new Person("Fuentes", 1));
        people.add(new Person("Ariel", 2));
        people.add(new Person("Jay", 3));

        System.out.println("START print list of people");

        //using comparable extends declared in Person class
        Collections.sort(people);

        //sort using java 7 anonymous class
        Collections.sort(people, new Comparator < Person > () {
            @Override
            public int compare(Person o1, Person o2) {
                // TODO Auto-generated method stub
                return o1.name.compareTo(o2.name);
            }
        });

        //sort using java 8 lambda
        Collections.sort(people, (o1, o2) - > o1.name.compareTo(o2.name));

        //Using for loop
        System.out.println("Using for loop");
        for (Person person: people) {
            System.out.println(person.name);
        }

        //Using lambda forEach
        System.out.println("Using lambda foreach");
        people.forEach(p - > System.out.println(p.name));

        //using streams
        System.out.println("Using lambda streams");
        people.stream()
            .filter(p - > p.name.startsWith("A"))
            .forEach(p - > System.out.println(p.name));

        System.out.println("END print list of people");
        //=============== MAP =======================
        System.out.println("\nMAP");

        //methods put(key, value), get(key), remove(key)
        //collection in key value pairs
        //HashMap(doesn't follow order), TreeMap(follows natural order), LinkedHashMap (follows order of
insertion)

        Map < Integer, String > myMap = new HashMap < Integer, String > ();
        myMap.put(1, "one");
        myMap.put(4, "four");
        myMap.put(2, "two");
        myMap.put(3, "three");

        Map < Integer, String > myMap2 = new TreeMap < > (myMap);

        //loop thru map using entrySet
        for (Map.Entry < Integer, String > maps: myMap.entrySet()) {
            System.out.println("Key=" + maps.getKey() + " Value=" + maps.getValue());
        }
```

```java
        //loop thru map using keySet
        for (Integer key: myMap2.keySet()) {
            System.out.println("Key=" + key + " Value=" + myMap.get(key));
        }

        //Hashtable is synchronized while HashMap is not synchronized
        //HashTable doesn't allow nulll key and values while hashMap allows null key and values


        //================ SET ===========
        System.out.println("\nSET");

        //methods add(element), remove(element) //not indexed
        //HashSet(doesn't follow order), TreeSet(follows natural order), LinkedHashSet(follows order of
insertion)
        Set < Integer > mySet = new HashSet < Integer > ();
        mySet.add(2);
        mySet.add(4);

        //for a set // [2, 4]
        for (Integer element: mySet) {
            System.out.println(element);
        }
        mySet.remove(2);

        Set < Integer > mySet2 = new HashSet < Integer > ();
        mySet2.add(3);
        mySet2.add(4);
        mySet2.add(5);
        mySet2.add(4);
        System.out.println("\n");

        //for a set // [3, 4, 5]
        for (Integer element: mySet2) {
            System.out.println(element);
        }

        //use to get common items between two sets (retainAll) or get unique items (removeAll)
        Set < Integer > intersection = new HashSet < Integer > (mySet);

        intersection.removeAll(mySet2);
        System.out.println(intersection);
    }
}
```

LAMBDA EXPRESSIONS

Lambdas Expression - are functions that doesn't belongs to the class and that exist in isolation. Not a method of a class.
Functions Benefits:

- Enables functional programming
- Readable and concise code

➢ Inline values

Int num = 9;
String word ="hi";

➢ *Function as values*

Workable printWord = () -> System.out.println("Hello") // java knows that this line refers to printWord abstract methods define in Workable interface.

Anonymous inner class – an inline implementation of an interface. a class that defines inside another class and doesn't have a name.

@FunctionalInterface **–** annotation used for interface that will be used for consuming a lambda expression

```
@FunctionalInterface
public interface Workable() {
  void printWord();
}
```

- **What is the Java Stream API and why is it useful?** The Java Stream API is used for processing collections of data in a functional programming style. It allows operations like filtering, mapping, and reducing, making data manipulation more concise and efficient.

```
list.stream().map(item -> {
    new Object(item.field);
})..collect(Collectors.toList());
```

SOLID PRINCIPLES

The following five concepts make up our SOLID principles:

- **Singles Responsibility Principle** - a class should have only one reason to change. Class should have one thing to do/responsibility.
- **Open/Closed Responsibility** - open for extension and closed for modification.
  - example. : extending square, rectangle to shape abstract class instead of if else method if shape.
- **Liskov Substitution** - subtypes must be substitutable for their base types. Superclass should be replaceable by subclass.
- **Interface Segregation** - clients should not be forced to implement an interface they don't use. keep interfaces small as possible.
- **Dependency Inversion** - depend upon abstraction (interface) not upon concrete classes.

DESIGN PATTERN

- **What is the Singleton design pattern in Java?** The Singleton pattern ensures that a class has only one instance and provides a global point of access to it. This is typically implemented with a private constructor and a static instance variable.

OTHERS

- **API** – application programming interface – a software/code written or developed by someone that can be used by others to integrate /connect to an application.
- **Framework** is like a blueprint that provides a predefined structure and set of tools for developing software applications. It sets the foundation, thereby simplifying the process of writing code and focusing on the unique aspects of your application.

The technical **difference** between a **framework and library** lies in a term called inversion of control. When you use a **library**, you are in charge of the flow of the application. You are choosing when and where to call the **library**. When you use a **framework**, the **framework** is in charge of the flow.